

25/9/2025

AT

Desenvolvimento de Serviços com SpringBoot

Professor(a): Flávio da Silva Neves

LINK GITHUB

- **Branch main – não tem Autenticação**

https://github.com/faculdade-infnet/V-1-Spring_Boot/tree/main/AT

- **Branch postgresql – está completo**

https://github.com/faculdade-infnet/V-1-Spring_Boot/tree/posgresql/AT

1. LISTE 4 TIPOS DE REQUISIÇÕES HTTP QUE O SPRING BOOT SUPORTA. DETALHE O FUNCIONAMENTO DE CADA UMA DELAS.

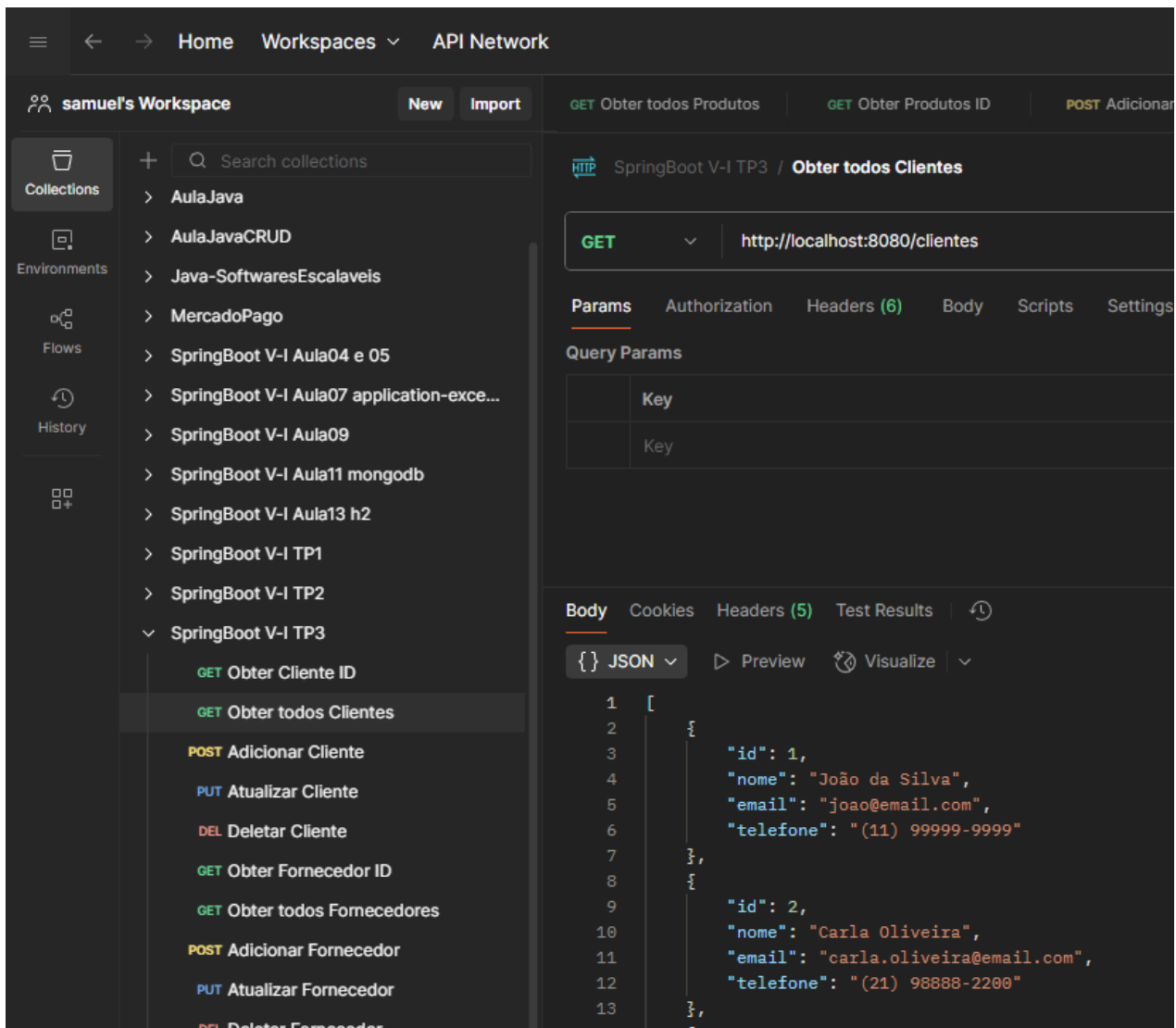
- **GET**

Usado para OBTER algum recurso hospedado no servidor.

Código:

```
// GET - Obtém todos os Clientes
@GetMapping
public ResponseEntity<List<Cliente>> getAll() {
    return ResponseEntity.ok().body(service.getAll());
}
```

POSTMAN

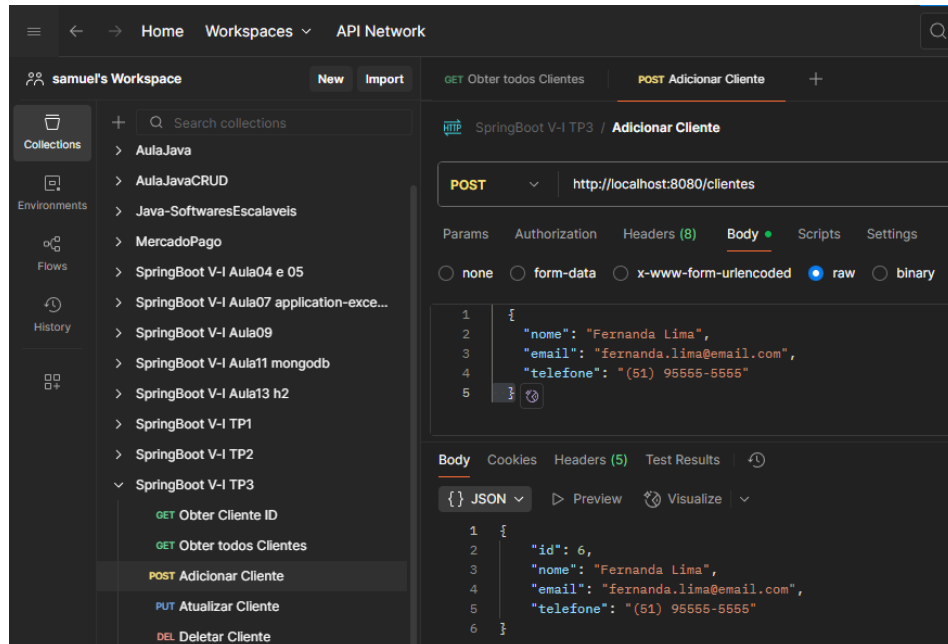


- POST

Usado para ENVIAR dados para o servidor.

```
// POST - Criação de Cliente
@PostMapping
public ResponseEntity<Cliente> create(@RequestBody Cliente obj) {
    return ResponseEntity.status(HttpStatus.CREATED).body(service.create(obj));
}
```

POSTMAN

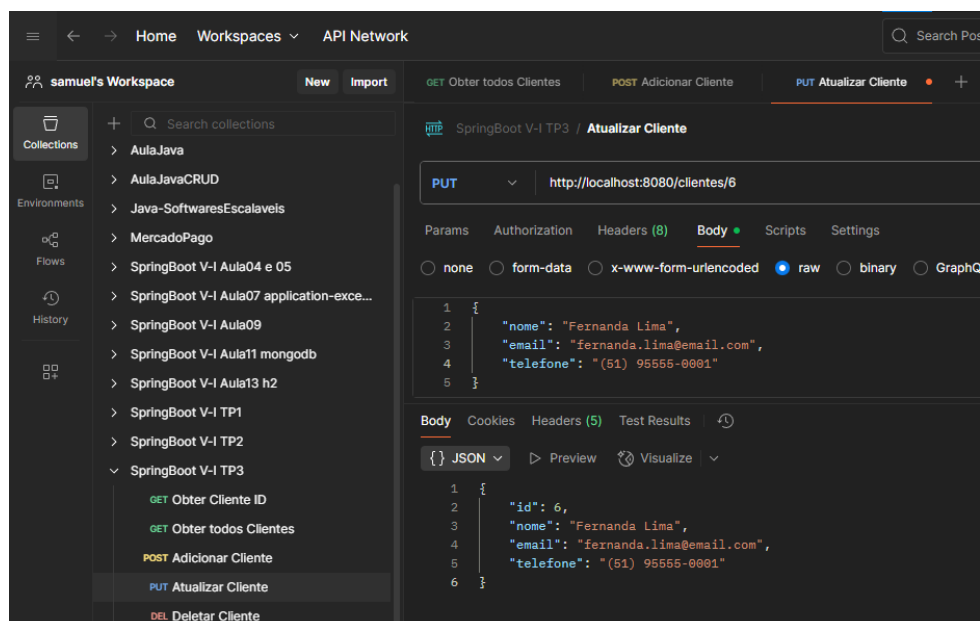


- PUT

Usado para ATUALIZAR dados de um recurso no servidor, geralmente através de um ID.

```
// PUT - Atualiza um Cliente através do id
@PutMapping(value =("/{id}")
public ResponseEntity<Cliente> update(@PathVariable Long id, @RequestBody Cliente
obj) {
    obj.setId(id);
    return ResponseEntity.ok().body(service.update(obj));
}
```

POSTMAN

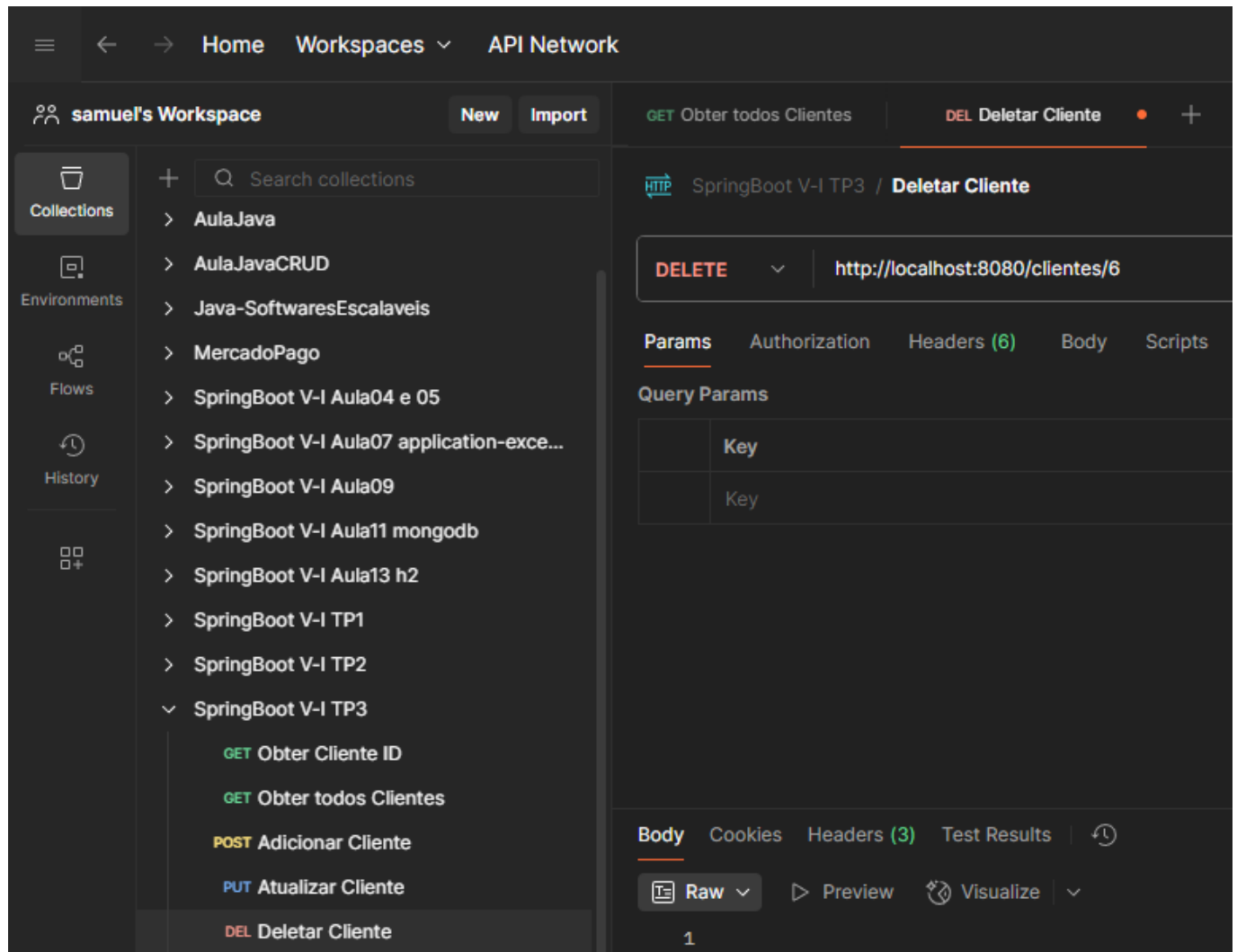


- DELETE

Usado para DELETAR um recurso no servidor, geralmente através de um ID.

```
// DELETE - Deleta um Cliente através do id
@DeleteMapping(value =("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}
```

POSTMAN



2. QUAIS AS PRINCIPAIS SEMELHANÇAS ENTRE OS BANCOS DE DADOS REDIS E MONGODB?

Redis e MongoDB são NoSQL, flexíveis, escaláveis, de alta performance, replicáveis e bem integrados a ecossistemas modernos, embora cada um seja otimizado para um caso de uso diferente (Redis: mais voltado para cache e dados temporários e MongoDB mais para armazenamento persistente de documentos).

- **NoSQL**

Ambos são classificados como bancos de dados NoSQL, ou seja, não seguem rigidamente o modelo relacional tradicional (tabelas, linhas, colunas).

- **Alta performance**

Foram projetados para alta velocidade em operações de leitura e escrita, Redis principalmente em memória e MongoDB com grandes volumes de documentos principalmente.

- **Escalabilidade horizontal**

Tanto Redis quanto MongoDB oferecem mecanismos de sharding (particionamento de dados) e replicação, permitindo escalar para lidar com grandes quantidades de dados e múltiplos usuários simultâneos.

- **Suporte a dados flexíveis**

Ambos permitem maior flexibilidade do que bancos SQL rígidos trabalhando com:

- **Open Source e grande comunidade**

Ambos têm versões open source e versões enterprise, além de ampla documentação e suporte em comunidades.

- **Integração com aplicações modernas**

Muito usados em arquiteturas de microserviços, aplicações web e mobile.

Possuem bibliotecas para diversas linguagens (Java, Python, C#, Node.js, etc.).

- **Replicação e tolerância a falhas**

Redis e MongoDB oferecem mecanismos de replicação de dados para garantir alta disponibilidade e recuperação em caso de falha.

3. QUAIS AS PRINCIPAIS DIFERENÇAS ENTRE OS BANCOS DE DADOS REDIS E MONGODB?

Modelo de dados	Banco chave-valor.	Banco orientado a documentos, baseado em JSON/BSON.
Armazenamento principal	In-memory (dados ficam na memória RAM, podendo persistir opcionalmente em disco).	Persistente em disco
Velocidade	Usado para cache e dados temporários.	Armazenamento permanente e grandes volumes de dados.
Persistência	Opcional.	Persistente por padrão.
Consultas	Operações simples por chave ou estruturas de dados.	Possui linguagem de consultas avançada (filtros, agregações, buscas textuais, geoespaciais, etc.).
Escalabilidade	Suporta replicação e sharding, geralmente usado em clusters menores ou como complemento a outro banco.	Sharding massivo, suportando grandes bancos distribuídos globalmente.
Complexidade de dados	Ótimo para dados pequenos e rápidos.	Documentos grandes, aninhados, coleções massivas e relacionamentos via <i>embeds</i> ou <i>references</i> .
Uso de memória	Mais intensivo, pois os dados vivem na RAM.	Mais econômico, pois os dados ficam em disco e usam RAM como cache secundário.
Comunicação	Protocolos binários otimizados, baixa latência.	Interface baseada em JSON/BSON, mais “pesada”.

4. QUAL FRAMEWORK PARA AUTOMAÇÃO DE TESTES O SPRING BOOT USA PARA TRATAR OS TESTES UNITÁRIOS? DESCREVA SEU FUNCIONAMENTO.

JUnit 5 o framework padrão de testes unitários em Java.

Funcionamento:

```
@Test
void testCreateCliente() throws Exception {
    when(service.create(any(Cliente.class))).thenReturn(cliente1);

    mockMvc.perform(post("/clientes")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(cliente1)))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.nome").value("João da Silva"));

    verify(service, times(1)).create(any(Cliente.class));
}
```

- **when(service.create(any(Cliente.class))).thenReturn(cliente1);**

Quando o mock service receber uma chamada create(...) com qualquer instância de Cliente (any(Cliente.class)), deve retornar o objeto cliente1 (pré-criado no teste).

- **mockMvc.perform(post("/clientes"))**

Inicia a simulação de uma requisição HTTP do tipo POST para o endpoint /clientes. mockMvc simula o dispatcher do Spring MVC sem subir servidor HTTP real.

- **.contentType(MediaType.APPLICATION_JSON)**

Define o header Content-Type: application/json. Importante porque o controller provavelmente espera JSON e usa @RequestBody para desserializar.

- **.content(objectMapper.writeValueAsString(cliente1)))**

O corpo da requisição é o JSON gerado a partir de cliente1 (pela ObjectMapper do Jackson). Assim o controller receberá um JSON igual ao objeto cliente1.

- **.andExpect(status().isCreated())**

Verifica que a resposta HTTP do controller foi 201 Created. Indica que o recurso foi criado com sucesso.

- **.andExpect(jsonPath("\$.nome").value("João da Silva"));**

Verifica o corpo da resposta (JSON) com JSONPath: \$.nome pega o campo nome da raiz do objeto JSON e exige que seu valor seja "João da Silva".

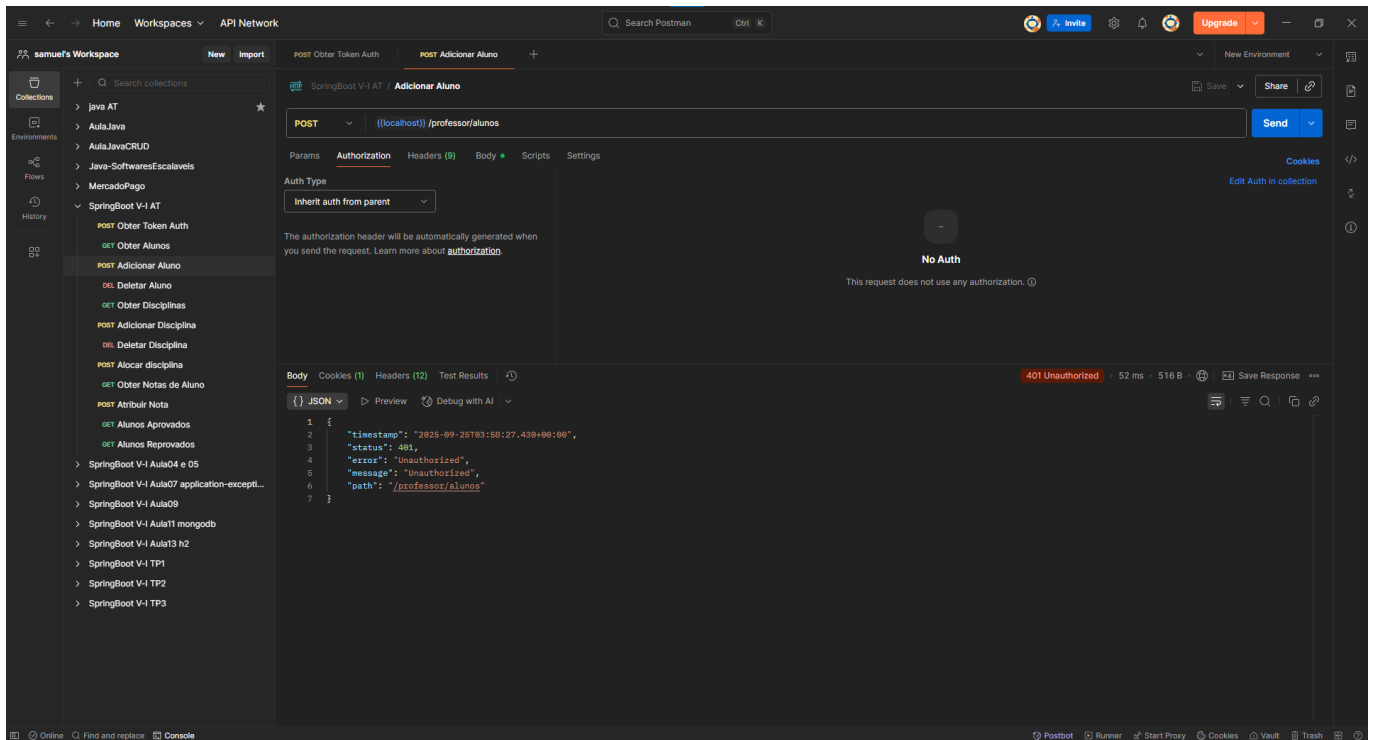
- **verify(service, times(1)).create(any(Cliente.class));**

Verificação de interação: confirma que service.create(...) foi chamado apenas uma vez durante a execução do controller.

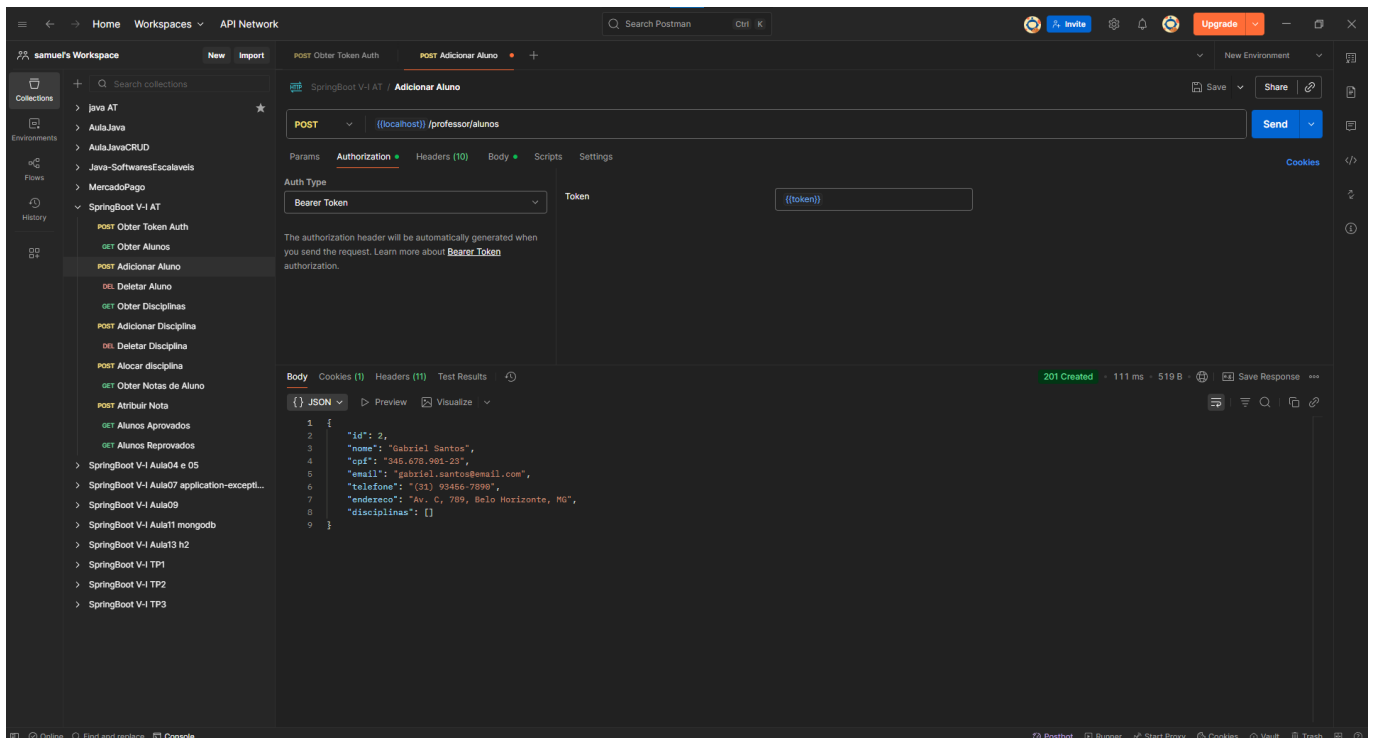
5. NO ASSESSMENT, IREMOS IMPLEMENTAR UM SERVIÇO COM AS SEGUINTE FUNCIONALIDADES:

1) Cadastrar Aluno

- Não autorizado



- Autorizado Token



Autorizado Body

The screenshot shows the Postman interface with a workspace named "samuel's Workspace". The left sidebar lists various API collections, including "SpringBoot V-1 AT". The main panel displays a POST request to "http://localhost:8080/professor/alunos". The request body is a JSON object representing a student:

```
1 {
2   "nome": "Gabriel Santos",
3   "cpf": "345.678.901-23",
4   "email": "gabriel.santos@email.com",
5   "telefone": "(31) 93456-7890",
6   "endereco": "Av. C, 789, Belo Horizonte, MG"
7 }
```

The response is a 201 Created status with a response time of 111 ms and a body size of 519 B. The response body is a JSON object:

```
1 {
2   "id": 2,
3   "nome": "Gabriel Santos",
4   "cpf": "345.678.901-23",
5   "email": "gabriel.santos@email.com",
6   "telefone": "(31) 93456-7890",
7   "endereco": "Av. C, 789, Belo Horizonte, MG",
8   "disciplinas": []
9 }
```

2) Cadastrar Disciplina

The screenshot shows the Postman interface with a workspace named "samuel's Workspace". The left sidebar lists various API collections, including "SpringBoot V-1 AT". The main panel displays a POST request to "http://localhost:8080/professor/disciplinas". The request body is a JSON object representing a discipline:

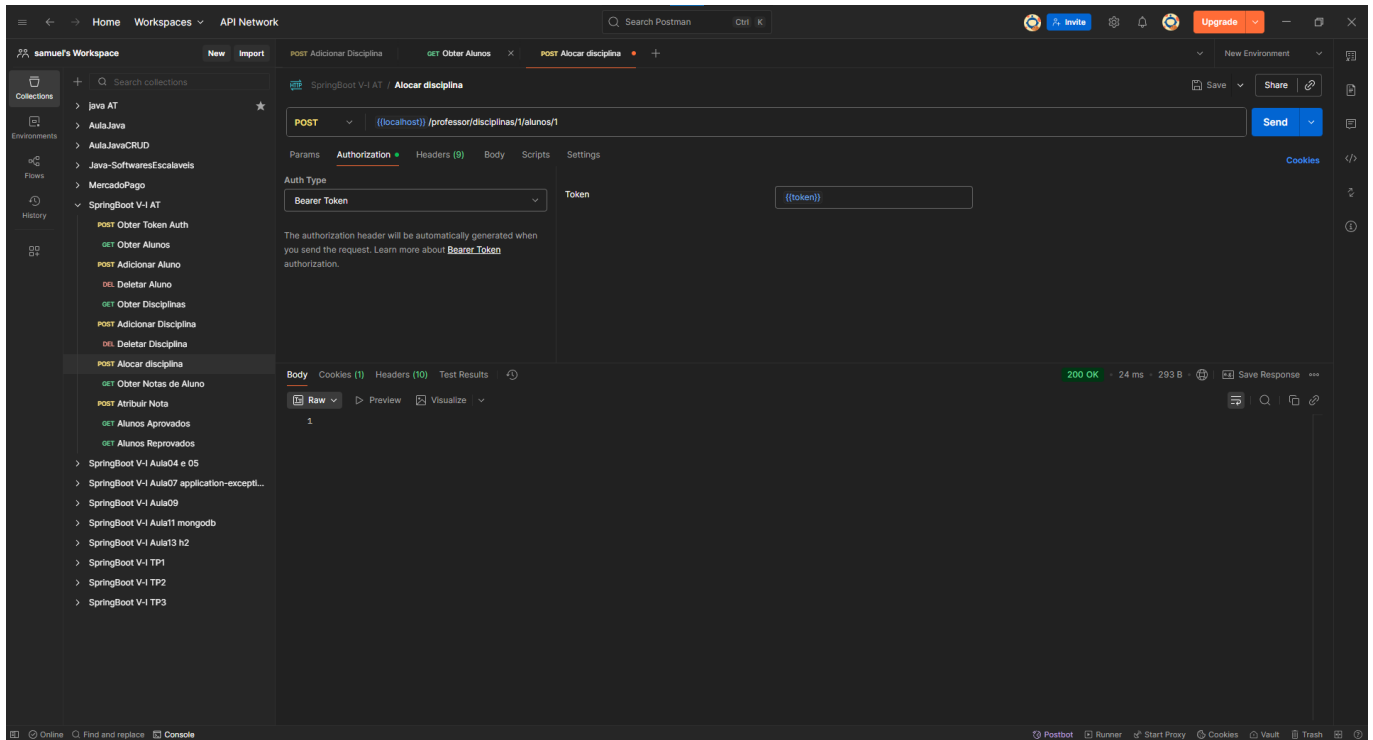
```
1 {
2   "nome": "Matemática",
3   "codigo": "01"
4 }
```

The response is a 201 Created status with a response time of 15 ms and a body size of 382 B. The response body is a JSON object:

```
1 {
2   "id": 1,
3   "nome": "Matemática",
4   "codigo": "01"
5 }
```

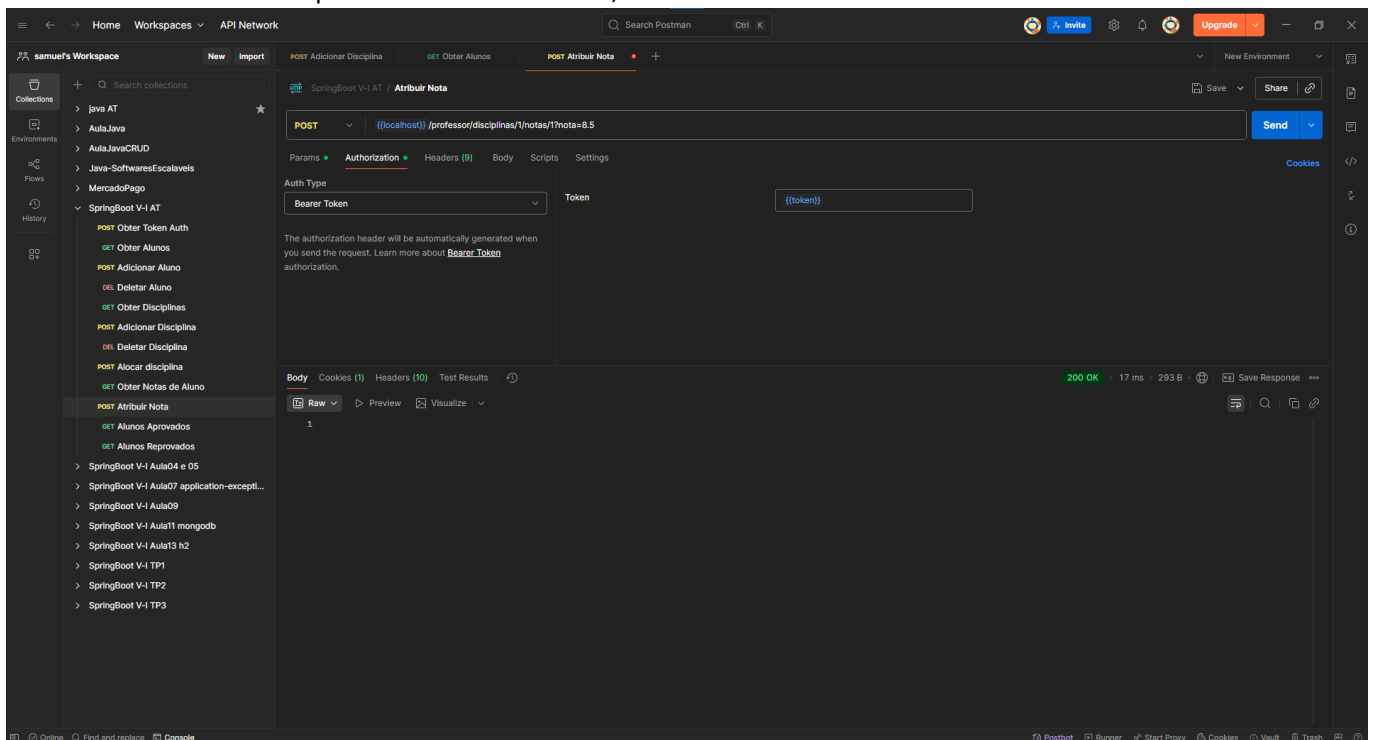

3) Alocar aluno em disciplina

- Aluno com id=1 em disciplina com id=1



4) Atribuir Nota

- Atribuir nota disciplina id = 1 e aluno id = 1, nota 8.5



- Consultar notas do aluno id=1

The screenshot shows the Postman interface for a workspace named 'samuel's Workspace'. The selected collection is 'SpringBoot V-1 AT'. The request is a GET to 'Obter Notas de Aluno' at the endpoint '((localhost)) /professor/alunos/1/notes'. The Authorization tab is active, showing 'Bearer Token' with the token '[[token]]'. The response is a 200 OK with a status of 8 ms and 369 B. The response body is a JSON object:

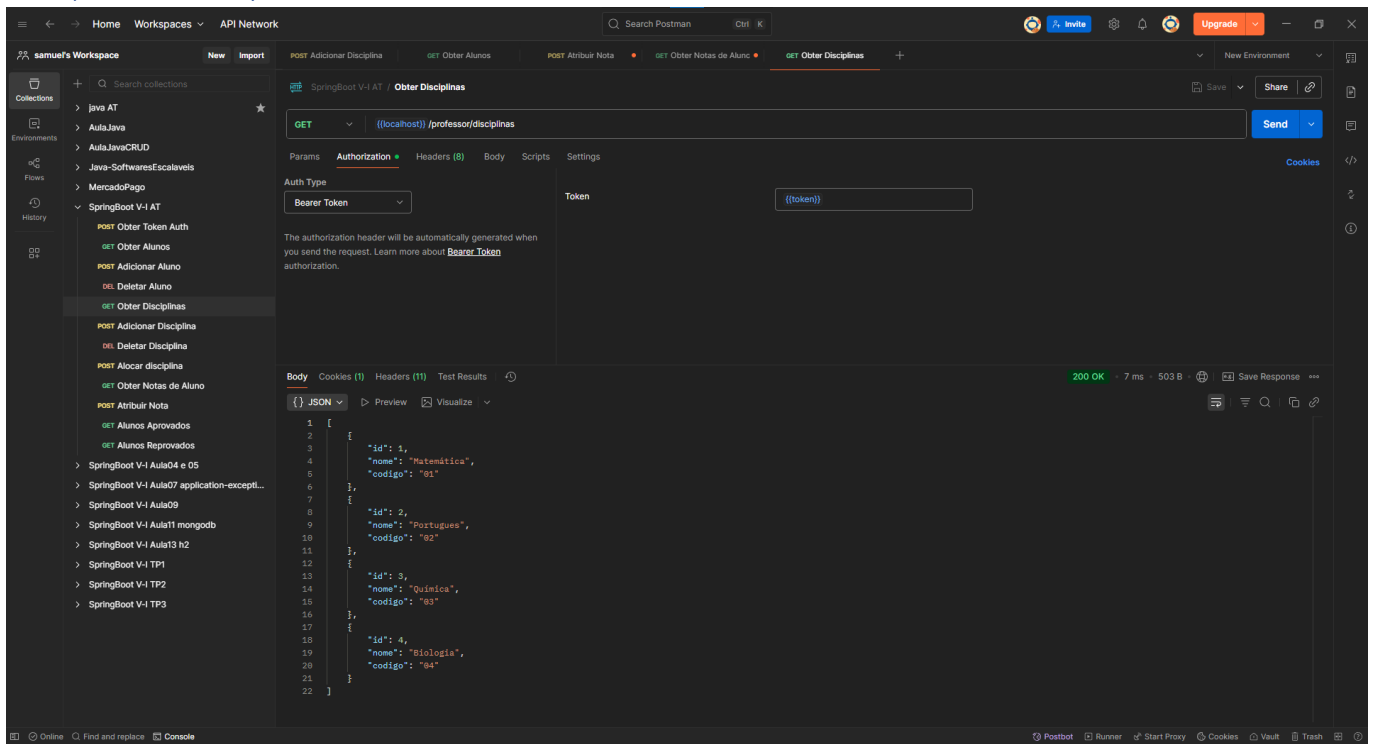
```
{
  "Matemática": 8.5,
  "Portugues": 6.0
}
```

5) Listar Alunos

The screenshot shows the Postman interface for the same workspace. The selected collection is 'SpringBoot V-1 AT'. The request is a GET to 'Obter Alunos' at the endpoint '((localhost)) /professor/alunos'. The Authorization tab is active, showing 'Bearer Token' with the token '[[token]]'. The response is a 200 OK with a status of 10 ms and 1.01 KB. The response body is a JSON array of student objects:

```
[
  {
    "id": 1,
    "nome": "Lucas Silva",
    "cpf": "123.456.789-01",
    "email": "lucas.silva@email.com",
    "telefone": "(11) 91234-5678",
    "endereco": "Rua A, 123, São Paulo, SP",
    "disciplinas": [
      {
        "id": 1,
        "nome": "Matemática",
        "codigo": "61"
      },
      {
        "id": 2,
        "nome": "Portugues",
        "codigo": "62"
      }
    ]
  },
  {
    "id": 2,
    "nome": "Gabriel Santos",
    "cpf": "345.678.901-23",
    "email": "gabriel.santos@email.com",
    "telefone": "(31) 93456-7890",
    "endereco": "Av. C, 789, Belo Horizonte, MG",
    "disciplinas": [
      {
        "id": 2,
        "nome": "Portugues",
        "codigo": "62"
      }
    ]
  }
]
```

6) Listar Disciplinas

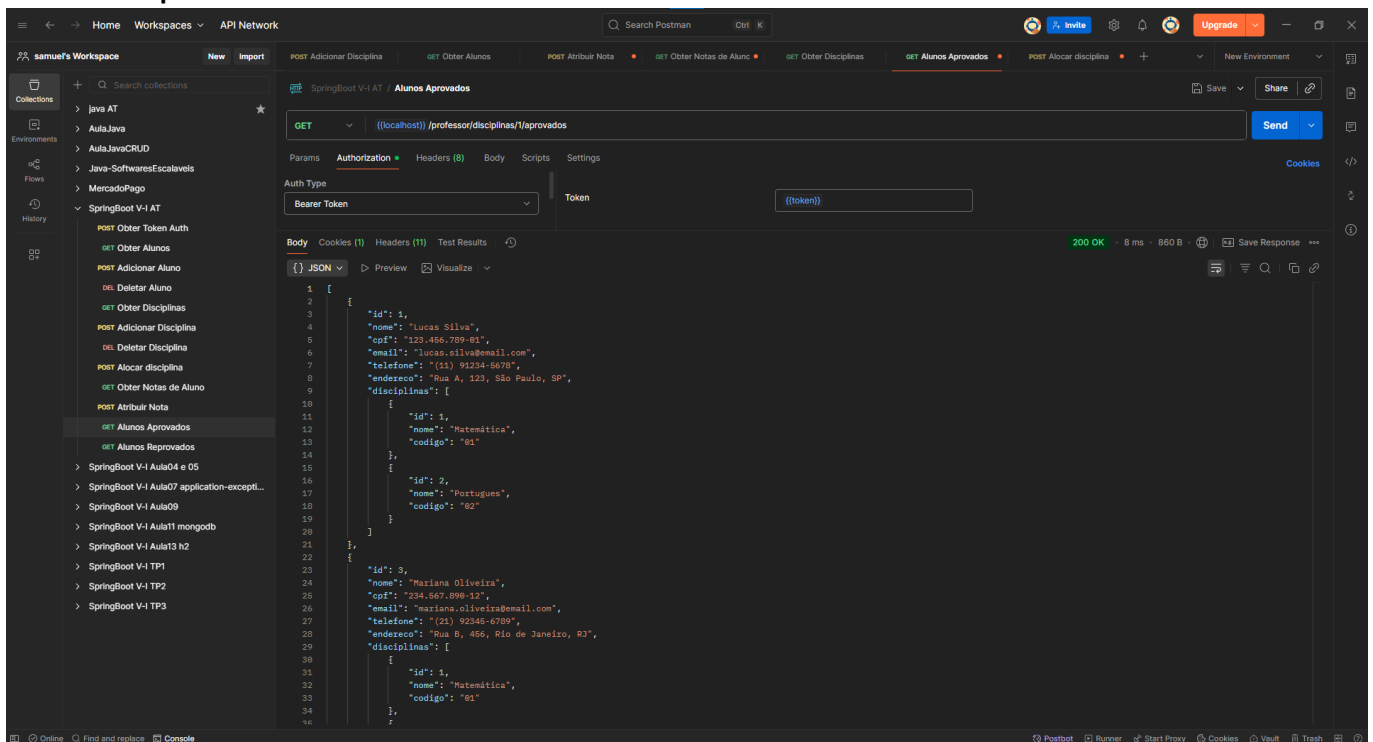


The screenshot shows a Postman interface with a workspace named "samuel's Workspace". The selected collection is "SpringBoot V-4 AT". The request is a GET to "((localhost)) /professor/disciplinas". The response is a 200 OK status with a JSON body containing an array of 4 disciplines.

```
1 [
2   {
3     "id": 1,
4     "nome": "Matemática",
5     "codigo": "01"
6   },
7   {
8     "id": 2,
9     "nome": "Portugues",
10    "codigo": "02"
11  },
12  {
13    "id": 3,
14    "nome": "Química",
15    "codigo": "03"
16  },
17  {
18    "id": 4,
19    "nome": "Biologia",
20    "codigo": "04"
21  }
22 ]
```

7) Alunos aprovados em uma determinada disciplina

- **Disciplina id = 1**

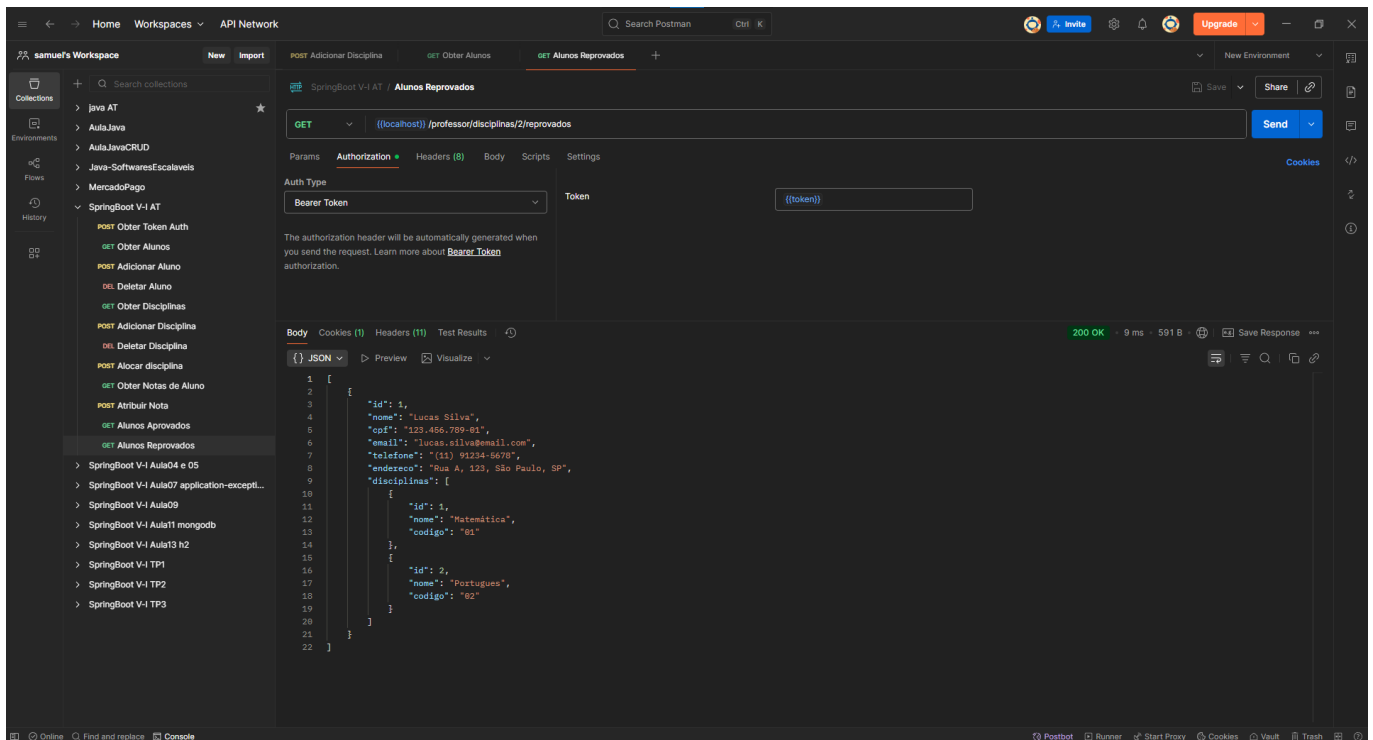


The screenshot shows a Postman interface with a workspace named "samuel's Workspace". The selected collection is "SpringBoot V-4 AT". The request is a GET to "((localhost)) /professor/disciplinas/1/aprovados". The response is a 200 OK status with a JSON body containing an array of 3 approved students.

```
1 [
2   {
3     "id": 1,
4     "nome": "Lucas Silva",
5     "cpf": "123.456.789-01",
6     "email": "lucas.silva@email.com",
7     "telefone": "(11) 91234-5678",
8     "endereco": "Rua A, 123, São Paulo, SP",
9     "disciplinas": [
10      {
11        "id": 1,
12        "nome": "Matemática",
13        "codigo": "01"
14      },
15      {
16        "id": 2,
17        "nome": "Portugues",
18        "codigo": "02"
19      }
20    ]
21  },
22  {
23    "id": 3,
24    "nome": "Mariana Oliveira",
25    "cpf": "234.567.890-12",
26    "email": "mariana.oliveira@email.com",
27    "telefone": "(21) 92345-6789",
28    "endereco": "Rua B, 456, Rio de Janeiro, RJ",
29    "disciplinas": [
30      {
31        "id": 1,
32        "nome": "Matemática",
33        "codigo": "01"
34      }
35    ]
36  }
37 ]
```

8) Alunos reprovados em uma determinada disciplina

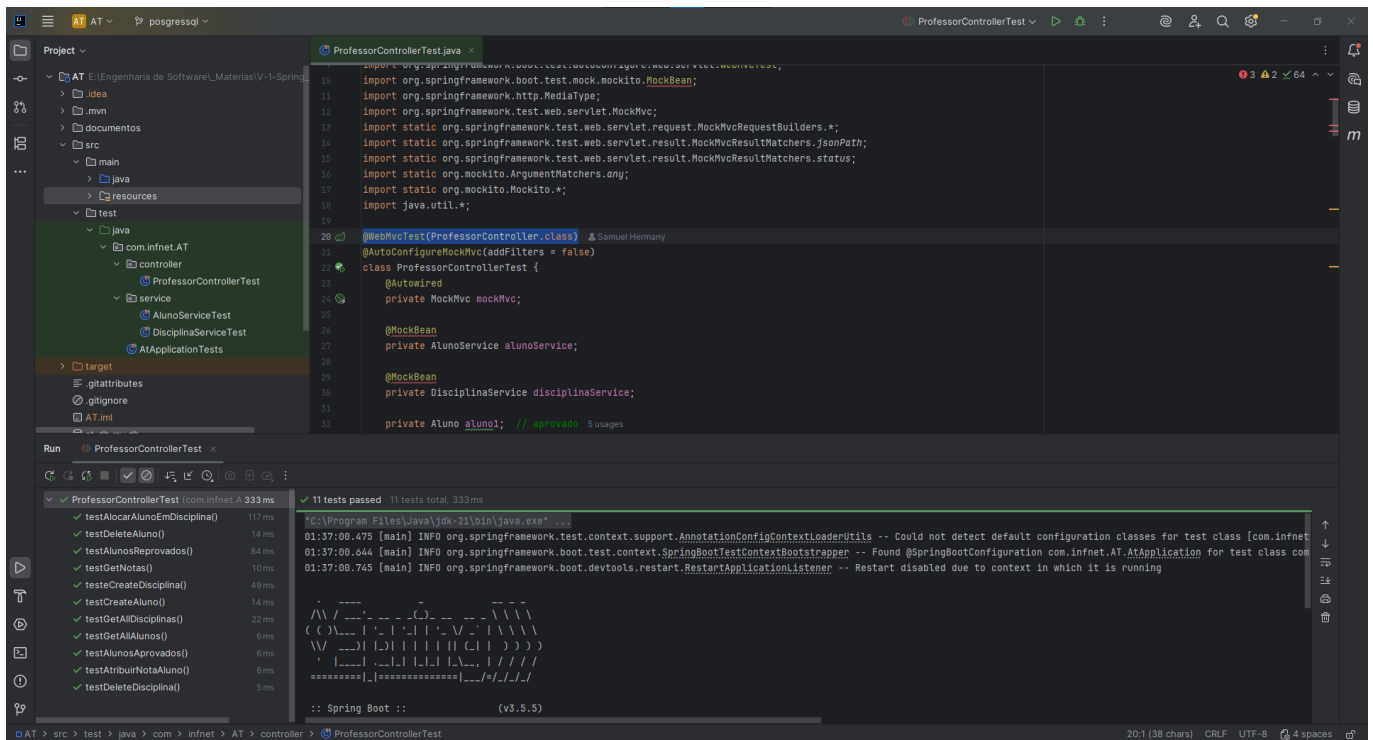
- **Disciplina id = 2**



9) Testes unit\u00e1rios para todos os m\u00e9todos

Crie arquivos **json** na pasta documentos para os testes, facilita o processo de testes no **Postman** ou **Insomnia**, al\u00e9m disso na pasta documentos tem tamb\u00e9m um arquivo com todas as rotas para teste no **Postman**.

- **ProfessorControllerTest**



AlunoServiceTest

The screenshot shows an IDE with the `AlunoServiceTest.java` file open. The code defines a test class `AlunoServiceTest` with the following structure:

```
package com.infnet.AT.service;

import ...

class AlunoServiceTest {
    @Mock
    private AlunoRepository alunoRepository;

    @InjectMocks
    private AlunoService alunoService;

    private Aluno aluno1;
    private Aluno aluno2;
    private Disciplina disciplina1;
    private Disciplina disciplina2;

    @BeforeEach
    void setup() {
        MockitoAnnotations.openMocks(this);
        // create alunos
        // mock repository methods
        aluno1 = new Aluno(1L, nome: "Lucas Silva", email: "123.456.789-01",
            email: "lucas.silva@gmail.com", telefone: "(11) 91234-5678",
            endereco: "Rua 8 - 194 - São Paulo - SP", new HashCode());
    }
}
```

The Run tab shows the test results for `AlunoServiceTest`:

- 4 tests passed, 4 tests total, 958 ms
- testDeleteAluno() 941 ms
- testCreateAluno() 9 ms
- testGetAluno() 3 ms
- testGetNotasPorAluno() 5 ms

The console output shows warnings from Mockito and Java HotSpot(TM) 64-Bit Server VM, indicating that the inline-mock-maker is being used and that dynamic loading of agents will be disallowed by default in a future release.

DisciplinaServiceTest

The screenshot shows an IDE with the `DisciplinaServiceTest.java` file open. The code defines a test class `DisciplinaServiceTest` with the following structure:

```
package com.infnet.AT.service;

import ...

class DisciplinaServiceTest {
    @Mock
    private DisciplinaRepository disciplinaRepository;

    @Mock
    private AlunoRepository alunoRepository;

    @InjectMocks
    private DisciplinaService disciplinaService;

    private Aluno aluno1;
    private Aluno aluno2;
    private Disciplina disciplina1;
    private Disciplina disciplina2;

    @BeforeEach
    void setup() {
        MockitoAnnotations.openMocks(this);
        // create disciplinas
        // mock repository methods
    }
}
```

The Run tab shows the test results for `DisciplinaServiceTest`:

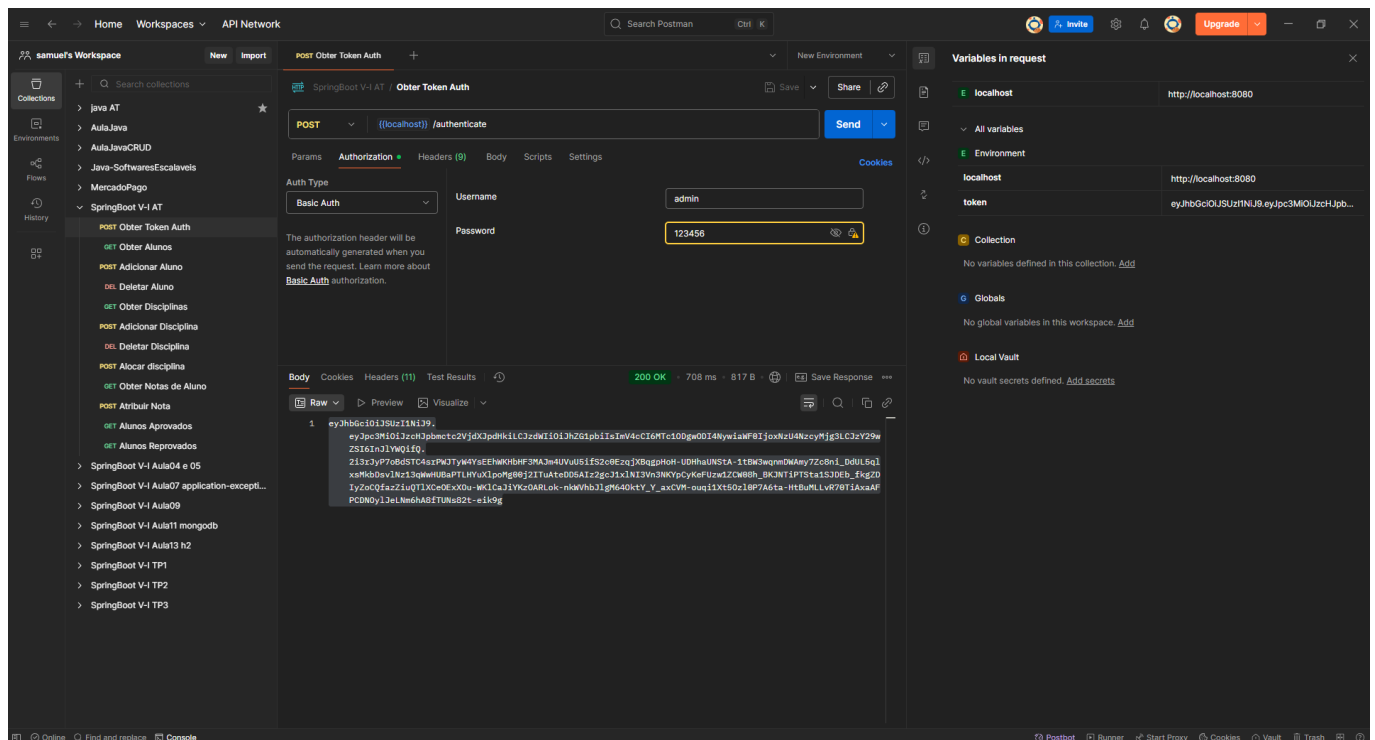
- 7 tests passed, 7 tests total, 1 sec 522 ms
- testAlocarAluno() 1 sec 487 ms
- testListarAlunosAprovados() 6 ms
- testListarAlunosReprovados() 4 ms
- testCreateDisciplina() 4 ms
- testAtribuirNotaAluno() 3 ms
- testGetAlunoDisciplina() 3 ms
- testDeleteDisciplina() 5 ms

The console output shows warnings from Mockito and Java HotSpot(TM) 64-Bit Server VM, indicating that the inline-mock-maker is being used and that dynamic loading of agents will be disallowed by default in a future release.

- **Site pra encriptação**

<https://bcrypt-generator.com/>

- **Obter Token**

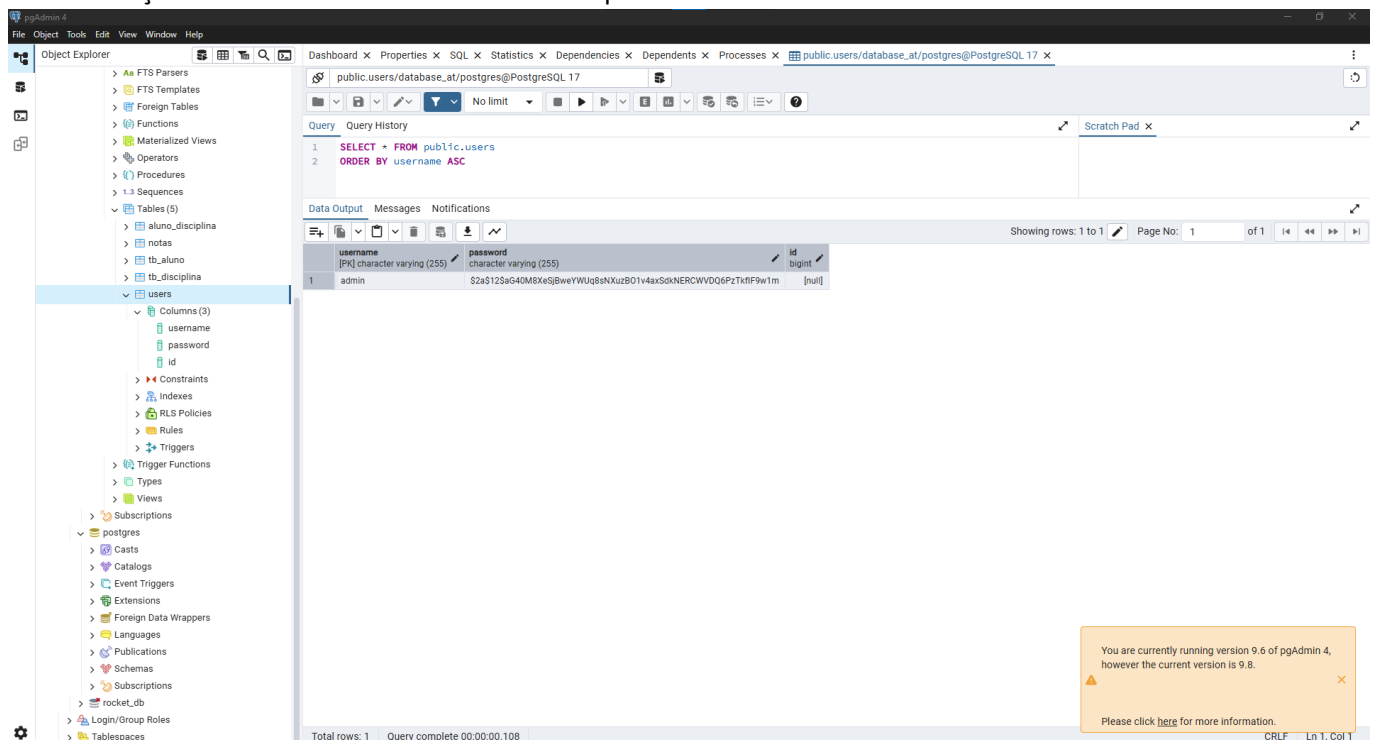


- pg admin

user: admin | senha: 123456

Token é salvo como uma variável pra facilitar os testes.

Endereço também é salvo como uma variável para facilitar os testes.



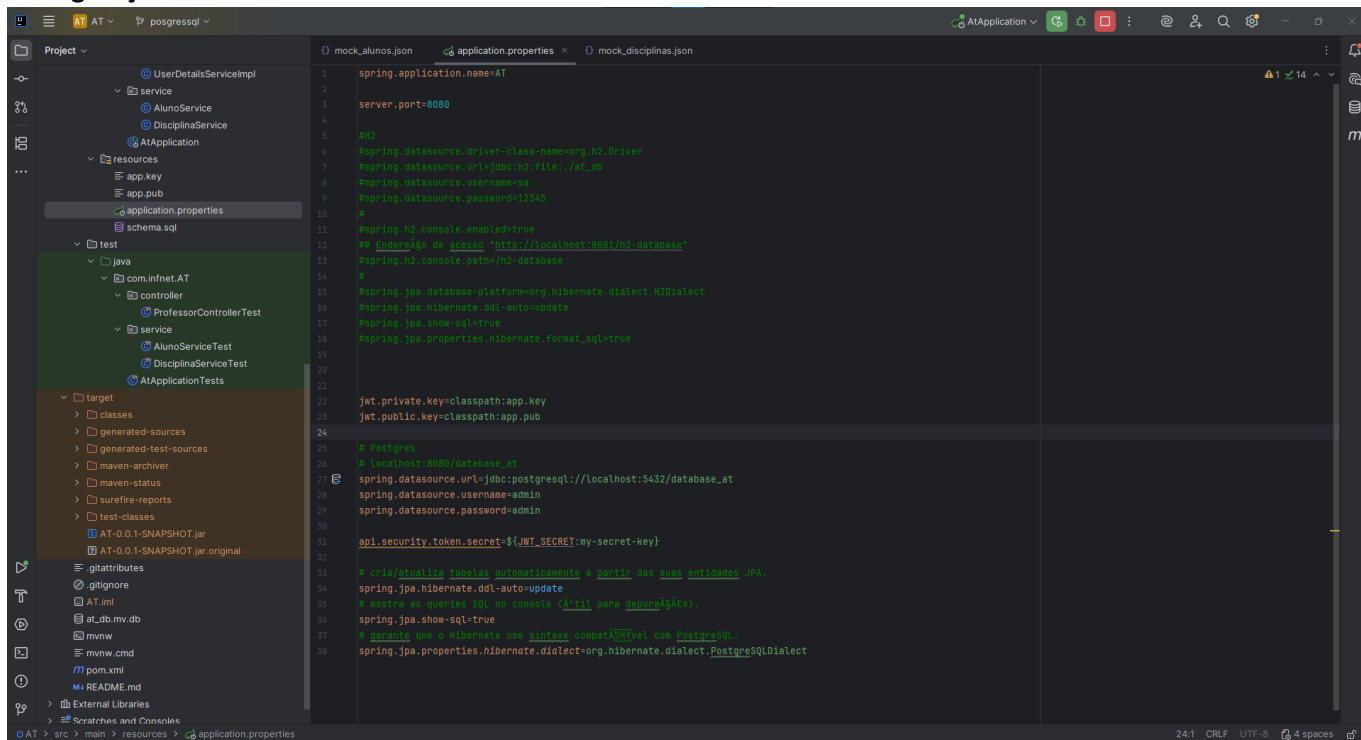
11) Banco de Dados escolhido

Fiz o projeto em dois bancos de dados para praticar mais, primeiro fiz em H2 porque é mais rápido pra min e depois fiz em postreSql.

O projeto está em duas branch:

- A branch **main** está com H2, tem tudo menos **autenticação e** arquivo ***.jar**.
- A branch **postressql** está em posgreSql, **tem tudo (projeto, rotas, testes e autenticação)**.

• Configuração do banco de Dados



```
spring.application.name=AT
server.port=8080

#H2
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.url=jdbc:h2:file:/at_db
spring.datasource.username=sa
spring.datasource.password=12345
#
spring.h2.console.enabled=true
# Indicação de acesso 'http://localhost:8081/h2-database'
spring.h2.console.path=/h2-database
#
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

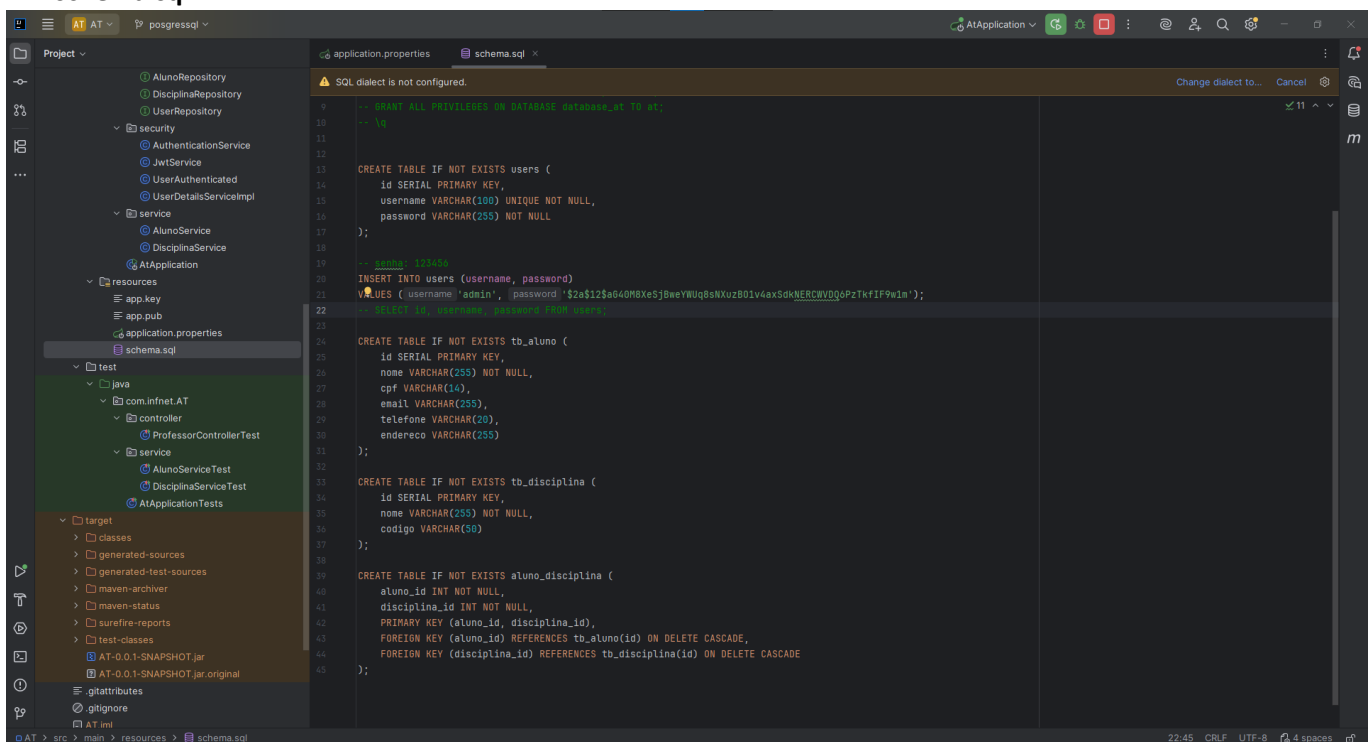
jwt.private.key=classpath:app.key
jwt.public.key=classpath:app.pub

# Postgres
# localhost:5432/database_at
spring.datasource.url=jdbc:postgresql://localhost:5432/database_at
spring.datasource.username=admin
spring.datasource.password=admin

api.security.token.secret=${JWT_SECRET:my-secret-key}

# Criar/tabela/tabela automaticamente e permitir que o usuário atualizado .PA
spring.jpa.hibernate.ddl-auto=update
# Permitir que o usuário atualizado (tudo para atualizar)
spring.jpa.show-sql=true
# Permitir que o usuário atualizado (tudo para atualizar)
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

• schema.sql



```
-- GRANT ALL PRIVILEGES ON DATABASE database_at TO at;
-- !@

CREATE TABLE IF NOT EXISTS users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL
);

-- admin: 123456
INSERT INTO users (username, password)
VALUES ('admin', '2a8125a640MBx5j8weYUq8sNXz801v4x5dkNERCWDQ0PzKfIF9w1n');
-- !@123456 username: password FROM users;

CREATE TABLE IF NOT EXISTS tb_aluno (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  cpf VARCHAR(14),
  email VARCHAR(255),
  telefone VARCHAR(20),
  endereco VARCHAR(255)
);

CREATE TABLE IF NOT EXISTS tb_disciplina (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  codigo VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS aluno_disciplina (
  aluno_id INT NOT NULL,
  disciplina_id INT NOT NULL,
  PRIMARY KEY (aluno_id, disciplina_id),
  FOREIGN KEY (aluno_id) REFERENCES tb_aluno(id) ON DELETE CASCADE,
  FOREIGN KEY (disciplina_id) REFERENCES tb_disciplina(id) ON DELETE CASCADE
);
```

6. FAÇA O DEPLOY DO SERVIÇO/APLICAÇÃO DESENVOLVIDO.

Gerei o arquivo ***.jar** conforme informado em aula.

